

Lecture 5: Basics of C Programming for HPC

Objective:

To understand and apply C programming fundamentals essential for building efficient High-Performance Computing (HPC) applications.

Duration: 1 Hour

Learning Outcomes

By the end of this session, students will be able to:

- Understand C language syntax, data types, and variables
- Apply control structures like loops and conditionals
- Define and call functions with arguments
- Recognize the importance of C in HPC systems

◆ 1. Introduction to C for HPC

- C is a **low-level**, **high-performance**, and **portable** language.
- It is widely used in **scientific computing**, **MPI programming**, and **system-level HPC code**.

2. Basics of C Programming

Data Types in C

Type	Description	Example
int	Integer (4 bytes)	<code>int x = 10;</code>
float	Floating point (4 bytes)	<code>float pi = 3.14;</code>
char	Character (1 byte)	<code>char ch = 'A';</code>
double	Double-precision float	<code>double d = 2.71828;</code>

📌 Variables and Operators

```
int a = 5, b = 10;  
int sum = a + b;
```

Operators:

- Arithmetic: +, -, *, /, %
- Relational: ==, !=, >, <, >=, <=
- Logical: &&, ||, !

📌 3. Control Flow Statements

✅ If Statement

```
int x = 15;  
if (x > 10) {  
    printf("x is greater than 10");  
}
```

✅ Switch Statement

```
int option = 2;  
switch(option) {  
    case 1: printf("Option 1"); break;  
    case 2: printf("Option 2"); break;  
    default: printf("Invalid");  
}
```

📌 Loops

For Loop:

```
for (int i = 1; i <= 5; i++) {  
    printf("%d ", i);  
}
```

While Loop:

```
int i = 1;  
while (i <= 5) {  
    printf("%d ", i);  
    i++;  
}
```

```
}
```

Do-While Loop:

```
int i = 1;
do {
    printf("%d ", i);
    i++;
} while (i <= 5);
```



4. Functions in C



Function Declaration & Definition

```
int add(int a, int b); // Declaration

int add(int a, int b) { // Definition
    return a + b;
}
```



Calling a Function

```
int result = add(5, 10);
printf("Result: %d", result);
```

Types of arguments:

- Pass by value (default in C)
- No return vs. return functions



Why C for HPC?

- Close to hardware → Faster execution
- Explicit memory management (using pointers)
- Compatible with MPI/OpenMP for parallel programming



Classroom Activities

♦ Activity 1: Code Identification

Show a small C snippet and ask students to:

- Identify variables, data types
- Predict output

♦ Activity 2: Write & Run (if Lab Available)

Task: Write a program to input two numbers and return their product using a function.

♦ Activity 3: Think-Pair-Share

Question: Why is C preferred in supercomputing environments?

Summary

- C provides speed and control, critical for HPC applications.
- Key fundamentals include data types, control structures, and modular code via functions.
- Strong grasp of C is essential before moving to parallel libraries like MPI or OpenMP.

Homework/Assignment

Task: Write a C program to:

1. Take an integer input from the user
2. Check if it's even or odd using `if-else`
3. Create a function to return the square of the number

Submission Deadline: Before next class